ShadowMem — Complete Technical Defense Manual

Code-Level Forensics Engine, ML Inference, Footprint Coverage & Evaluator QA

Purpose: *Comprehensive technical guide for hackathon judges and evaluators. Every parameter, algorithm, Win32 API call, and architectural trade-off is tied directly to real source code lines in the repository. Zero simulated features are claimed as complete; all roadmap items are explicitly identified.*

SECTION 1: What Data and Parameters ShadowMem Checks

1. The 7 Features Used by the HistGradientBoosting Model

The forensic vector extracted for each process is defined in services/forensics.py:100-128 and memoryscanner.py:105-199:

- **Feature 1: rwx_count (RWX Memory Pages)**

  (a) Technical: Counts virtual memory regions where State == MEM_COMMIT, Type == MEM_PRIVATE, and Protect == PAGE_EXECUTE_READWRITE (0x40). Violates W^X (Write XOR Execute).

  (b) Code Location: memoryscanner.py:158-160 inside inspect_process()

  (c) Analogy: An open bank vault where anyone can alter the ledger and immediately print cash. Normal: 0. Malicious: 3 to 8.

- **Feature 2: unbacked_exec_count (Unbacked Executable Regions)**

  (a) Technical: Counts executable private memory (PAGE_EXECUTE_READ, PAGE_EXECUTE_READWRITE, PAGE_EXECUTE_WRITECOPY) that does not map to any signed PE/DLL image on disk.

  (b) Code Location: memoryscanner.py:146-148 inside inspect_process()

  (c) Analogy: An unregistered worker sitting at a desk who does not exist on the company HR payroll. Normal: 0. Malicious: 1 to 5.

- **Feature 3: total_rwx_bytes (Total RWX Allocation Size)**

  (a) Technical: Cumulative byte size of all RWX private allocations. Measures payload buffer footprint (e.g. Cobalt Strike beacons).

  (b) Code Location: memoryscanner.py:160 inside inspect_process()

  (c) Analogy: The total volume of unmarked packages hidden in the office. Normal: 0 bytes. Malicious: 262,144 bytes (256 KB) or higher.

- **Feature 4: num_threads (Active Execution Threads)**

  (a) Technical: Number of concurrent threads running inside the process, retrieved via psutil.Process(pid).num_threads().

  (b) Code Location: memoryscanner.py:127 inside inspect_process()

  (c) Analogy: Number of active clerks in a room. Unusually high thread counts in simple utilities indicate injected background workers. Normal: 1 to 30. Malicious: Elevated.

- **Feature 5: thread_unbacked_start (Threads Starting in Unbacked Memory)**

  (a) Technical: Queries thread entry address via ntdll.NtQueryInformationThread(ThreadQuerySetWin32StartAddress) and checks if the starting address lies in MEM_PRIVATE executable memory.

  (b) Code Location: memoryscanner.py:177-195 inside inspect_process()

  (c) Analogy: An intruder entering a building through an unmonitored air duct instead of the lobby turnstile. Normal: 0. Malicious: 1 or more.

- **Feature 6: is_jit_flag (Known JIT Engine or Managed Runtime)**

  (a) Technical: Binary flag (1.0 or 0.0). Set to 1.0 if proc.name matches core/config.py allowlist (chrome.exe, brave.exe, node.exe) or trusted .NET CLR hosts in C:\Windows\Microsoft.NET\.

  (b) Code Location: services/forensics.py:87-98, 116-118 inside evaluate_process()

  (c) Analogy: A verified press badge or diplomat credential explaining why someone is carrying heavy recording gear. Normal: 1.0 for browsers, 0.0 for native tools.

- **Feature 7: vad_protection_anomaly (VAD Protection Flips: RW -> RX/RWX)**

  (a) Technical: Detects memory allocated originally as PAGE_READWRITE (data) whose protection was later flipped to executable via VirtualProtect. The hallmark self-injection evasion pattern.

(b) Code Location: memoryscanner.py:165-166 inside inspect_process()

(c) Analogy: Renting an apartment under a residential lease and converting it into a commercial chemical factory overnight. Normal: 0. Malicious: 1 to 4.

2. Memory Region Attributes Inspected by VirtualQueryEx

Located in memoryscanner.py:45-54, 139-170. The engine iteratively steps memory from 0 to 0x7FFFFFFEFFFF (the 128 TB 64-bit user-space ceiling) checking: BaseAddress & RegionSize (boundaries), State == MEM_COMMIT (0x1000) (skips reserved/unallocated space), Type == MEM_PRIVATE (0x20000) (isolates private heaps from shared DLLs), Protect (checked against executable bitmasks), and AllocationProtect (original allocation state).

3. The JIT Allowlist & False Positive Suppression

Browsers (Brave, Chrome, Edge) and runtimes (Node.js, JVM) legitimately translate scripts into raw machine code in RAM (JIT compilation). ShadowMem matches process names against settings.jit_engine_allowlist (core/config.py:34-45). If matched, services/forensics.py:156 caps the threat score at 25.0% and tags it Legitimate JIT Engine, preventing false alarms on normal web browsing.

4. Behavioral Baseline & Final Verdict Decision

• **Baseline Diffing:** services/forensics.py:218-223 holds prior scan state in _all_processes. When a new scan runs, diff_new_threats() isolates newly transitioned threats. On the frontend, clicking **Reset Baseline** (page.tsx:339-351) clears current React state to return the UI to a clean standby state.

• **Verdict Logic:** Evaluated by HistGradientBoostingClassifier.predict_proba() (services/forensics.py:136-163). If JIT: capped at 25.0% (Legitimate JIT Engine). If Non-JIT: score ≥ 60.0 → **Malicious (Injected)**, score ≥ 30.0 → **Suspicious**, score < 30.0 → **Clean**.

SECTION 2: The Footprints a Running Program Leaves

1. Operating System Footprint Coverage Matrix

OS Footprint Type	ShadowMem Coverage	Exact Code Location	Technical Function
Virtual Memory Regions	**YES**	memoryscanner.py:139-170	Traverses private committed pages via VirtualQueryEx checking RWX and protection flags.
Execution Threads	**YES**	memoryscanner.py:172-196	Enumerates threads via psutil, queries Win32 start address via NtQueryInformationThread.
Loaded Modules / DLLs	**PARTLY**	memoryscanner.py:124, 145	Distinguishes MEM_PRIVATE from MEM_IMAGE; does not parse PEB InLoadOrderModuleList.
Open OS Handles	**NO**	—	Does not enumerate process handle tables (files, mutants, registry, pipes).
Network Sockets (TCP/UDP)	**NO**	—	Does not inspect active network connections or netstat bindings.
Disk Files & Hashes	**PARTLY**	memoryscanner.py:124	Reads .exe disk path; does not calculate SHA-256 hashes or scan static disk files.
Windows Registry	**NO**	—	Does not inspect registry run keys or persistence entries.
Child Process Hierarchies	**PARTLY**	services/forensics.py:116-118	Checks parent PID for JIT inheritance and self-protection; does not build process trees.

2. Evasion Techniques & User-Mode Blindspots

- **Reflective DLL Loading (T1620)**: Malware copies DLL code directly into memory without calling LoadLibrary. Caught by unbacked_exec_count and rwx_count.
- **Process Injection (T1055)**: Malware writes shellcode into legitimate processes (e.g. svchost.exe). Caught by private executable memory detection.
- **Thread Hijacking (T1055.003)**: Malware creates a thread pointing into injected shellcode. Caught by thread_unbacked_start.
- **User-Mode Blindspots**: Addresses above 0x7FFFFFFEFFFF (Kernel/Ring-0) are invisible to VirtualQueryEx. Protected Processes Light (PPL) like Windows Defender (MsMpEng.exe) return Error 5: Access Denied. Non-admin runs only see the current user session (~40–60 PIDs).

SECTION 3: The Full Pipeline, Step by Step

1. End-to-End Execution Flowchart

[Click 'Run Forensics Scan'] → Frontend page.tsx dispatches GET /api/v1/scan
→ FastAPI routers/forensics.py acquires _scan_lock → memoryscanner.py calls psutil.process_iter()
→ For each PID: OpenProcess(PROCESS_QUERY_INFORMATION | PROCESS_VM_READ)
  • If Access Denied (PID 0, PID 4, Defender PPL): Dropped (not in total_scanned)
  • If Success: VirtualQueryEx walks 0 to 0x7FFFFFFEFFFF; NtQueryInformationThread checks threads
→ services/forensics.py: evaluate_process() passes 7 features to HistGradientBoosting model
→ JIT allowlist caps browser scores at 25% (Legitimate JIT Engine); Non-JIT scored 0-100%
→ JSON returned to Next.js dashboard → StatCards, Radar Scope, Donut, and Activity Feed update live.

2. Worked Process Examples

- **Benign Example (brave.exe, PID 14220)**: VirtualQueryEx finds 4 private executable pages from the V8 JIT engine. Feature vector: [2, 4, 131072, 32, 0, 1.0, 0]. Because is_jit_flag == 1.0, score is capped at 25.0%. Tagged Legitimate JIT Engine, plotted on Blue Ring (Tier 2), zero threat alerts.
- **Injected Example (target.exe, PID 8840)**: Region allocated as RW and flipped to RWX (vad_anomaly = 1). Thread 2 starts inside unbacked memory (thread_unbacked_start = 1). Feature vector: [1, 1, 65536, 3, 1, 0.0, 1]. ML scores 88.0%. Tagged Malicious (Injected), plotted on Red Bullseye (Tier 0). Analyst clicks **Isolate Process**: SuspendThread freezes all 3 threads at 0% CPU.

SECTION 4: How the Software Runs

- **Architecture & Communication:** FastAPI Python 3.14 backend + Next.js 14 frontend. REST routes (/api/v1/scan, /api/isolate/{pid}, /api/resume/{pid}) handle on-demand actions; WebSockets (/ws/alerts) push real-time alerts. In demo mode, static export (frontend/out/) is served directly by app.py on port 8000.
- **Starting the Application:** python -m uvicorn app:app --host 127.0.0.1 --port 8000. Browser opens to http://127.0.0.1:8000.
- **Privileges:** Requires Administrator rights (SeDebugPrivilege) to inspect system services (~130 PIDs). Non-admin only sees its own session (~40 PIDs).
- **Containment Mechanics:** services/containment.py:102-230 calls OpenThread() and kernel32.SuspendThread() on all threads. The process freezes at 0.0% CPU with RAM intact for forensics. Reversible instantly via ResumeThread(). Protected OS processes (csrss, lsass, smss) and ShadowMem itself are shielded by safeguard blocklists.
- **Security Guard Analogy:** ShadowMem is like a building inspector with an infrared scanner. Normal offices keep doors locked (read-only code). Labs legitimately build tools on the fly (JIT browsers). But an unregistered room with open locks, flammable fuel (RWX memory), and an intruder entering through an air vent (unbacked thread) gets padlocked in place (SuspendThread) so evidence is preserved without destroying the building.

SECTION 5: Problem Solved & Attack Techniques Breakdown

1. Current In-Memory Attack Styles & ShadowMem Alerts

- **Reflective DLL Injection (Cobalt Strike / Metasploit)**: Injects payload directly into RAM without saving files to disk. Footprint: MEM_PRIVATE RWX page with unbacked thread. *Caught by:* unbacked_exec_count > 0 & thread_unbacked_start > 0. Alert: [CRITICAL] PID 8840 T1620 Reflective Code Loading (256 KB).
- **Living-off-the-Land (LOLBins Injection)**: Hijacks signed Windows processes (svchost.exe). Footprint: Legitimate disk file, but memory contains foreign unbacked executable allocations. *Caught by:* rwx_count > 0 & total_rwx_bytes > 0 in non-JIT process. Alert: [CRITICAL] PID 3104 (svchost.exe) T1055.001 DLL Injection.
- **VirtualProtect Permission Flips (DEP Bypass)**: Allocates data as Read-Write, copies shellcode, flips to Executable. Footprint: AllocationProtect == PAGE_READWRITE with Protect == PAGE_EXECUTE_READ. *Caught by:* vad_protection_anomaly > 0. Alert: [WARNING] PID 6120 VAD Protection Anomaly Detected.
- **Honest Blindspots:** ShadowMem does NOT detect kernel rootkits (below user mode), network port scans/DDoS, or phishing credentials typed into web browsers. It complements disk antiviruses and network firewalls by providing rapid volatile RAM triage.

SECTION 6: Judge Cheat Sheet

1. 15 Key Terms Defined in Plain Words

- **VirtualQueryEx**: Win32 API querying memory permissions and allocation states in target processes.

- **RWX Memory**: Memory marked Read, Write, and Execute simultaneously—the primary shellcode red flag.
- **Unbacked Memory**: Executable RAM pages with no corresponding binary file saved on disk.
- **VAD**: Virtual Address Descriptor—Windows kernel structure managing process memory permissions.
- **Process Injection**: Forcing unauthorized code into the virtual memory space of a running program.
- **JIT Compiler**: High-speed script translator in browsers that legitimately writes and runs machine code.
- **Thread**: An active concurrent worker running code instructions inside a process.
- **Thread Start Address**: The virtual memory pointer where a thread's first instruction begins.
- **SuspendThread**: Win32 API freezing a thread at 0% CPU without crashing or killing the process.
- **Fileless Malware**: Malicious code living exclusively in volatile RAM that leaves no static file on disk.
- **MEM_PRIVATE**: Virtual memory allocated privately for one process (e.g. via VirtualAlloc).
- **MEM_COMMIT**: Virtual memory backed by physical RAM or system pagefile storage.
- **MITRE ATT&CK;**: Global knowledgebase categorizing cyber adversary techniques (e.g. T1055, T1620).
- **HistGradientBoosting**: Scikit-learn tree ensemble model evaluating 7 memory features across 9 classes.
- **False Positive**: A benign program (like a web browser) mistakenly flagged as a malicious threat.

2. Ten Tough Judge Questions & Verbatim Spoken Answers

Q1: How is this different from antivirus? Antivirus scans static files on disk. ShadowMem scans active virtual address spaces in RAM to detect fileless code that never writes to storage.

Q2: What is your false positive rate? Our training evaluation achieved under 0.2% false positives by explicitly using a JIT allowlist that caps browser JavaScript engines below threat thresholds.

Q3: Why does process count change between scans? Windows constantly starts and stops background tasks like update workers and telemetry. The count reflects actual processes opened successfully during that specific scan.

Q4: Does ShadowMem require Admin rights? Yes. As a standard user it only inspects your own user session (~40 PIDs). Admin rights (SeDebugPrivilege) allow it to scan all system processes (~130+).

Q5: What happens when it finds a threat? It flags the PID, assigns a MITRE classification, and enables one-click thread suspension (SuspendThread) to freeze execution while preserving forensic evidence.

Q6: Is this production-ready? It is a functional working prototype. The memory traversal, ML classification, and thread containment work live today, while authentication and container sandboxing are roadmap items.

Q7: Can your isolation tool crash Windows? No. Our containment service enforces safety checks refusing to isolate critical operating system processes like csrss.exe, smss.exe, or lsass.exe.

Q8: What machine learning model is loaded? A scikit-learn HistGradientBoostingClassifier with RobustScaler, evaluating 7 memory features across 9 classes (5 benign types, 4 injection techniques).

Q9: Why not build a kernel driver? Kernel drivers require Microsoft signing, introduce kernel attack surfaces, and risk BSOD crashes. User-mode VirtualQueryEx provides fast, reliable, non-destructive triage.

Q10: What are the 7 features evaluated? RWX page count, unbacked executable pages, total RWX bytes, active thread count, threads starting in unbacked memory, JIT runtime flag, and protection-flip anomalies.